Notes on Type Theory

Apiros3

First Version : Oct 16, 2025
Last Update : – –, 2025

Contents

1 Introduction **2**

2 Judgements and Contexts **2**

3 Dependent Type Theory **2**
3.1 Basic Definitions 2

4 Universes **3**

5 Propositional Truncation **3**

6 Algebra 'Zoo' **3**
6.0.1 Algebra 3

7 Other (move later) **4**
7.1 Intrinsic vs Extrinsic 4
7.2 Theory of Signatures 5
7.3 Semantics 5
7.3.1 Algebra 5
7.3.2 Morphisms 7
7.3.3 Displayed Algebras 7
7.4 Models of Type Theories 8
7.4.1 Categories with Families (CwFs) 8
7.4.2 Aside on substitutions 8

8 The Natural Numbers **8**

1 Introduction

- Introduction to Homotopy Type Theory by Egbert Rijke

2 Judgements and Contexts

A mathematical argument consists of a sequence of deductive steps, which can be represented by inference rules of the form

$$\frac{\mathcal{H}_1 \quad \mathcal{H}_2 \quad \dots \quad \mathcal{H}_n}{\mathcal{C}}$$

Inference rules contain a finite list $\mathcal{H}_1, \mathcal{H}_2, \dots, \mathcal{H}_n$ of judgement for the premises (hypothesis) and a single judgement $\mathcal{C}$ for the conclusion.

3 Dependent Type Theory

3.1 Basic Definitions

We outline the basic definitions of MLTT (Martin-Löf Type Theory).

Definition 3.1.1. *MLTT consists of four* ***judgements****:*

1. *A is a well-formed type in context Γ, written*

$$\Gamma \vdash A \text{ type}$$

2. *A and B are judgementally equal types in context Γ, written*

$$\Gamma \vdash A \doteq B \text{ type}$$

3. *a is an element of type A in context Γ, written*

$$\Gamma \vdash a : A$$

4. *a and b are judgementally equal elements of type A in context Γ, written*

$$\Gamma \vdash a \doteq b : A$$

Definition 3.1.2. *A* ***context*** *is a (potentially empty) finite list of variable declarations $x_1 : A_1, x_2 : A_2(x_1), \dots, x_n : A_n(x_1, \dots, x_{n-1})$ such that for each $1 \leq k \leq n$, we can derive the judgement*

$$x_1 : A_1, \dots, x_{k-1} : A_{k-1}(x_1, \dots, x_{k-2}) \vdash A_k(x_1, \dots, x_{k-1}) \text{ type}$$

using the inference rules of type theory. Variable names can be chosen freely so long as there is no overlap.

Definition 3.1.3. *Consider a type A in context Γ. A* ***family of types*** *over A in context Γ is a type $B(x)$ in context $\Gamma, x : A$. That is, when*

$$\Gamma, x : A \vdash B(x) \text{ type}$$

we say that B is a family of types over A in context Γ. Alternatively, $B(x)$ is a type ***indexed*** *by $x : A$ in context Γ.*

Definition 3.1.4. *Let B be a type family over A in context Γ. A **section** of the family B over A in context Γ is an element of type $B(x)$ in context $\Gamma, x : A$, as in the judgement*

$$\Gamma, x : A \vdash b(x) : B(x)$$

Then, we say that b is a section of the family B over A in context Γ. Alternatively, $b(x)$ is the element of type $B(x)$ indexed by $x : A$ in context Γ.

Definition 3.1.5. *The inference rules for the formation of contexts, types, and their elements are given by,*

$$\frac{\Gamma, x : A \vdash B(x)\ \text{type}}{\Gamma \vdash A\ \text{type}} \qquad \frac{\Gamma \vdash A \doteq B\ \text{type}}{\Gamma \vdash A\ \text{type}} \qquad \frac{\Gamma \vdash A \doteq B\ \text{type}}{\Gamma \vdash B\ \text{type}}$$

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash A\ \text{type}} \qquad \frac{\Gamma \vdash a \doteq b : A}{\Gamma \vdash a : A} \qquad \frac{\Gamma \vdash a \doteq b : A}{\Gamma \vdash b : A}$$

Definition 3.1.6. *The inference rules that postulate judgemental equality as an equivalence relation is given as follows:*

$$\frac{\Gamma \vdash A\ \text{type}}{\Gamma \vdash A \doteq A\ \text{type}} \qquad \frac{\Gamma \vdash B \doteq A\ \text{type}}{\gamma \vdash A \doteq B\ \text{type}} \qquad \frac{\Gamma \vdash A \doteq B\ \text{type} \qquad \Gamma \vdash B \doteq C\ \text{type}}{\Gamma \vdash A \doteq C\ \text{type}}$$

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash a \doteq a : A} \qquad \frac{\Gamma \vdash a \doteq b : A}{\Gamma \vdash b \doteq a : A} \qquad \frac{\Gamma \vdash a \doteq b : A \qquad \Gamma \vdash b \doteq c : A}{\Gamma \vdash a \doteq c : A}$$

4 Universes

A

5 Propositional Truncation

6 Algebra 'Zoo'

6.0.1 Algebra

The computation for the types of algebras is given by the standard interpretation into the `Set` universe. The operations are defined inductively with notation $-^A$, overloaded for `Con`, `Ty`, and `Tm` as follows:

$$\begin{aligned}
-^A &: \mathtt{Ty} \to \mathtt{Set} \to \mathtt{Set} \\
\iota^A &\equiv X \\
(\iota \to A)^A \quad X &\equiv X \to A^A\ X
\end{aligned}$$

$$\begin{aligned}
(-)^A &: \mathrm{Con} \to \mathbf{Set} \to \mathbf{Set} \\
\Gamma^A X &\equiv \{A : \mathrm{Ty}\} \to \mathrm{Var}\,\Gamma\,A \to A^A X \\
(-)^A &: \mathrm{Tm}\,\Gamma\,A \to \{X : \mathbf{Set}\} \to \Gamma^A X \to A^A X \\
(\mathrm{var}\,x)^A\,\gamma &\equiv \gamma\,x \\
(\mathrm{app}\ t\ u)^A\,\gamma &\equiv t^A\,\gamma\,(u^A\,\gamma) \\
(-)^A &: \mathrm{Sub}\,\Gamma\,\Delta \to \{X : \mathbf{Set}\} \to \Gamma^A X \to \Delta^A X \\
\sigma^A\,\gamma\,x &\equiv (\sigma\,x)^A\,\gamma
\end{aligned}$$

7 Other (move later)

7.1 Intrinsic vs Extrinsic

Intrinsic constructions give sanity lemmas for free, by only allowing well-typed terms to be constructed. Here we give an intrinsic encoding of the lambda calculus such that only well-typed terms can be constructed. The typing rules become the constructors of the term datatype. We use De Bruijn indices (as we don't care about the implementation here).

The object types are given as

```
data Ty : Set where
    ι    : Ty
    _⇒_ : Ty → Ty → Ty
```

where we can define optional types as needed.

A context is a list of types that looks like

```
data Ctx : Set where
    ϵ   : Ctx
    _▷_ : Ctx → Ty → Ctx

data Var : Ctx → Ty → Set where
    vz : ∀ {Γ A}   → Var (Γ ▷ A) A              -- top
    vs : ∀ {Γ A B} → Var Γ A → Var (Γ ▷ B) A    -- weaken
```

Then `Var Γ A` is inhabited exactly when `A` occurs in Γ at the right index.

Then intrinsic terms are given by

```
data Tm : Ctx → Ty → Set where
    var : ∀ {Γ A} → Var Γ A → Tm Γ A
    lam : ∀ {Γ A B} → Tm (Γ ▷ A) B → Tm Γ (A ⇒ B)
    app : ∀ {Γ A B} → Tm Γ (A ⇒ B) → Tm Γ A → Tm Γ B
```

7.2 Theory of Signatures

Definition 7.2.1. *The **theory of signatures** or **ToS** is a small type theory equipped with:*

- *An empty base type ι*
- *A first-order function type $\iota \to -$ (with application but no λ-abstraction)*

The full syntax of this theory can be specified inductively as follows:

$$\begin{array}{lcllcl}
\mathrm{Ty} & : & \mathrm{Set} & \mathrm{Var} & : & \mathrm{Con} \to \mathrm{Ty} \to \mathrm{Set} \\
\iota & : & \mathrm{Ty} & \mathrm{vz} & : & \mathrm{Var}\,(\Gamma \triangleright A)\,A \\
\iota \to _ & : & \mathrm{Ty} \to \mathrm{Ty} & \mathrm{vs} & : & \mathrm{Var}\,\Gamma\,A \to \mathrm{Var}\,(\Gamma \triangleright B)\,A \\
 & & & & & \\
\mathrm{Con} & : & \mathrm{Set} & \mathrm{Tm} & : & \mathrm{Con} \to \mathrm{Ty} \to \mathrm{Set} \\
\bullet & : & \mathrm{Con} & \mathrm{var} & : & \mathrm{Var}\,\Gamma\,A \to \mathrm{Tm}\,\Gamma\,A \\
_ \triangleright _ & : & \mathrm{Con} \to \mathrm{Ty} \to \mathrm{Con} & \mathrm{app} & : & \mathrm{Tm}\,\Gamma\,(\iota \to A) \to \mathrm{Tm}\,\Gamma\,\iota \to \mathrm{Tm}\,\Gamma\,A
\end{array}$$

Here, `Con` contexts are lists of types and `Var` are well-typed De Bruijn indices in the usual sense.

Definition 7.2.2. ***Parallel substitutions*** *map variables to terms by*

$$\begin{aligned}
&\mathrm{Sub} : \mathrm{Con} \to \mathrm{Con} \to \mathrm{Set} \\
&\mathrm{Sub}\ \Gamma\ \Delta \equiv \{A : \mathrm{Ty}\} \to \mathrm{Var}\ \Delta\ A \to \mathrm{Tm}\ \Gamma\ A
\end{aligned}$$

The action of substitution on terms is given by

$$\begin{aligned}
&-[-] : \mathrm{Tm}\ \Delta\ A \to \mathrm{Sub}\ \Gamma\ \Delta \to \mathrm{Tm}\ \Gamma\ A \\
&(\mathrm{var}\ x)[\sigma] :\equiv \sigma\ x \\
&(\mathrm{app}\ t\ u)[\sigma] :\equiv \mathrm{app}\ (t[\sigma])\ (u[\sigma])
\end{aligned}$$

The identity substitution is then just var*. We also define composition by:*

$$\begin{aligned}
&- \circ - : \mathrm{Sub}\ \Delta\ \Xi \to \mathrm{Sub}\ \Gamma\ \Delta \to \mathrm{Sub}\ \Gamma\ \Xi \\
&(\sigma \circ \delta)\ x :\equiv (\sigma\ x)[\delta]
\end{aligned}$$

Example 7.2.3. Signatures for the natural numbers and binary trees can be written as follows:

$$\begin{array}{lcl}
\mathrm{NatSig} & :\equiv & \bullet \triangleright (\mathit{zero} : \iota) \triangleright (\mathit{suc} : \iota \to \iota) \\
\mathrm{TreeSig} & :\equiv & \bullet \triangleright (\mathit{leaf} : \iota) \triangleright (\mathit{node} : \iota \to \iota \to \iota)
\end{array}$$

7.3 Semantics

7.3.1 Algebra

The types of algebras are given by the standard interpretation into the Set universe. The operations are then defined inductively, overloaded for Con, Ty, and Tm.

Consider the mapping from Types to Sets, given a concrete set X to stand for the base type ι. Then, we interpret every type T to be the set $T^A\ X$ as follows:

$$\begin{aligned}
&-^A : \mathsf{Ty} \to \mathsf{Set} \to \mathsf{Set} \\
&\iota^A \qquad\quad X :\equiv X \\
&(\iota \to A)^A \quad X :\equiv X \to A^A\ X
\end{aligned}$$

Intuitively, a type is a space of values, and choosing X fixes what the value of a base type is. The interpretation of the function is then just all functions between the sets the interpret the domain and codomain.

The interpretation of contexts map it to their respective environments. That is, we have

$$
\begin{aligned}
&-^A : \mathsf{Con} \to \mathsf{Set} \to \mathsf{Set} \\
&\Gamma^A\ X :\equiv \{A : \mathsf{Ty}\} \to \mathsf{Var}\ \Gamma\ A \to A^A\ X
\end{aligned}
$$

Given a type A and a variable x of type A in Γ (equivalently $x \in \mathsf{Var}\ \Gamma\ A$), the environment γ returns a semantic value $\gamma(x) \in A^A\ X$. That is, $\Gamma^A\ X$ is all possible assignments of actual values to variables listed in Γ respecting their types.

The interpretation of terms correspond to an evaluation of the term given an environment. The context tells how to evaluate the free variables, and otherwise we work recursively down. That is,

$$
\begin{aligned}
&-^A : \mathsf{Tm}\ \Gamma\ A \to \{X : \mathsf{Set}\} \to \Gamma^A\ X \to A^A\ X \\
&(\mathsf{var}\ x)^A \quad\ \gamma := \gamma\ x \\
&(\mathsf{app}\ t\ u)^A \quad \gamma := t^A\ \gamma\ (u^A\ \gamma)
\end{aligned}
$$

Substitutions can then be interpreted as reindexing of environments as

$$
\begin{aligned}
&-^A : \mathsf{Sub}\ \Gamma\ \Delta \to \{X : \mathsf{Set}\} \to \Gamma^A\ X \to \Delta^A\ X \\
&\sigma^A\,\gamma\,x := (\sigma\,x)^A\,\gamma
\end{aligned}
$$

Explicitly, given a $\gamma \in \Gamma^A\,X$ and a variable $x \in \mathsf{Var}\,\Delta\,A$, the new environment value at x is the evaluation of $\sigma\,x$ under γ. Intuitively, substitutions act by evaluating the terms being substituted under the current assignments.

Definition 7.3.1. *Define* ***algebras*** *as follows:*

$$
\begin{aligned}
&\mathsf{Alg} : \mathsf{Con} \to \mathsf{Set}_1 \\
&\mathsf{Alg}\,\Gamma :\equiv (X : \mathsf{Set}) \times \Gamma^A\,X
\end{aligned}
$$

Example 7.3.2. We have for example $\mathsf{Alg}\,\mathsf{NatSig} \equiv (X : \mathsf{Set}) \times (zero : X) \times (suc : X \to X)$.

Fix the interpretation of the base type. The current version is similar to that of STLC, but we will move towards a version that is dependent. Intuitively, we have

- The **base space** is the set of environments Γ^A
- Over each environment $\gamma \in \Gamma^A$ is a fiber that may depend on γ
- A **type** in context assigns the right fiber to each γ
- A **term** is a way to pick for every γ, one element in the corresponding fiber
- A **substitution** moves one between the base spaces, and reindexing acts like a pull-back along that map.

Then, we can take

$$(\Gamma : \mathsf{Con})^A : \mathsf{Set}$$

which represents the set of all environments for Γ. Each $\gamma \in \Gamma^A$ assigns semantic values to the variables declared in Γ.

Types depend on contexts, and so the interpretation is given by

$$(A : \mathsf{Ty}\,\Gamma)^A : \Gamma^A \to \mathsf{Set}$$

is a family of sets varying with γ. For each environment γ is a set $A^A\,\gamma$ (the fiber over γ).

Intuitively, as the meaning of a dependent type depends on the earlier variables, when we have $n : \mathbb{N}$ inside Γ, then taking $A \equiv \mathsf{Vec}\,X\,n$, A is interpreted as the set of length n lists of X. The fiber $A^A\,\gamma$ is just vectors of length $\gamma(n)$.

Terms are then just sections of the bundle $A \to \Gamma^A$. That is, for each γ, a point $t^A\,\gamma$ in the fiber $A^A\,\gamma$.

7.3.2 Morphisms

Morphisms correspond to functions between underlying sets that preserves all specified structure. The interpretation for types is given by

$$\begin{array}{ll} -^M : (A : \mathsf{Ty})\,\{X_0\,X_1 : \mathsf{Set}\}\,(X^M : X_0 \to X_1) \to A^A\,X_0 \to A^A\,X_1 \to \mathsf{Set} \\ \iota^M & X^M\,\alpha_0\,\alpha_1 :\equiv X^M\,\alpha_0 = \alpha_1 \\ (\iota \to A)^M & X^M\,\alpha_0\,\alpha_1 :\equiv (x : X_0) \to A^M\,X^m\,(\alpha_0\,x)\,(\alpha_1\,(X^M\,x)) \end{array}$$

We fix the interpretation for base types as X_0 and X_1 and consider the function $X^M : X_0 \to X_1$. Then, A^M specifies exactly what it means for X^M to preserve an operation with type A. At the base type, this corresponds to equality. At the first-order function type, preservation is a quantified statement over X_0 (which is like saying the functions preserve structure for any argument from the domain).

7.3.3 Displayed Algebras

»» Displayed algebras correspond to the dependent algebras, where every displayed algebra component lies above the corresponding components in the base algebra. ««

Fix some predicate on the base set $X^D : X \to \mathsf{Set}$. The displayed algebra specifies what it means to be 'related' with respect to that predicate.

The displayed algebra on types looks like

$$\begin{array}{ll} -^D : (A : \mathsf{Ty})\,\{X\} \to (X \to \mathsf{Set}) \to A^A\,X \to \mathsf{Set} \\ \iota^D & X^D\,\alpha :\equiv X^D\,\alpha \\ (\iota \to A)^D & X^D\,\alpha \equiv (x : X)\,(x^D : X^D\,x) \to A^D\,X^D\,(\alpha\,x) \end{array}$$

On a ground type, elements $\alpha : X$ are related exactly when they satisfy the base predicate. On function types, it is related if and only if it maps related inputs to related outputs.

The displayed algebra for contexts looks like

$$\begin{array}{l} -^D : (\Gamma : \mathsf{Con})\{X\} \to (X \to \mathsf{Set}) \to \Gamma^A\,X \to \mathsf{Set} \\ \Gamma^D\,X^D\,\gamma :\equiv \{A : \mathsf{Ty}\}(x : \mathsf{Var}\,\Gamma\,A) \to A^D\,X^D\,(\gamma\,x) \end{array}$$

A context is related when the mapping function from variables to values is related pointwise for every possible variable $x : \mathsf{Var}\,\Gamma\,A$.

7.4 Models of Type Theories

7.4.1 Categories with Families (CwFs)

Definition 7.4.1. *A **category with families** (cwf) [Dybjer 1995] is a specific category with a terminal structure, alongisde family structure. Here, objects are* $\mathsf{Con} : \mathsf{Set}$ *and morphisms are* $\mathsf{Sub} : \mathsf{Con} \to \mathsf{Con} \to \mathsf{Set}$. *The terminal object* $\bullet$ *has a unique morphism* $\epsilon : \mathsf{Sub}\,\Gamma\,\bullet$. *The family structure contains* $\mathsf{Ty} : \mathsf{Con} \to \mathsf{Set}$ *and* $\mathsf{Tm} : (\Gamma : \mathsf{Con}) \to \mathsf{Ty}\,\Gamma \to \mathsf{Set}$.

In initial models of type theories, objects correspond to typing contexts, and morphisms to parallel substitutions, and the terminal object to the empty context. Ty is a presheaf over the category of contexts and Tm is a displayed presheaf over Ty.

7.4.2 Aside on substitutions

Fix a small category $\mathcal{C}$ and write $\widehat{\mathcal{C}} := \mathcal{C}^{\mathrm{op}} \to \mathsf{Set}$. We consider contexts to be objects $\Gamma \in \widehat{\mathcal{C}}$ and substitutions are natural transformations $\sigma : \Delta \Rightarrow \Gamma$.

Now consider the slice $\widehat{\mathcal{C}}/\Gamma$. Objects are pairs (A, p) where $p : A \to \Gamma$ is a morphism in $\widehat{\mathcal{C}}$.

Now define $\mathsf{Ty}(\Gamma)$ to be the objects of the slice $\widehat{\mathcal{C}}/\Gamma$, where objects are morphisms $p : A \to \Gamma$ in $\widehat{\mathcal{C}}$ (so a natural transformation $A \Rightarrow \Gamma$), and morphism between $p : A \to \Gamma$ and $q : A' \to \Gamma$ is a morphism $f : A \to A'$ in $\widehat{\mathcal{C}}$ such that $p = q \circ f$. Hence,

In particular, a type over Γ is a display map $p : A \Rightarrow \Gamma$ in $\widehat{\mathcal{C}}$. So, for each $C \in \mathcal{C}$, we have a function $p_C : A(C) \to \Gamma(C)$ that is natural in C.

8 The Natural Numbers